

11. Grafikus felület specifikációja

48 – GonoszOnosz

Konzulens:

Marcsingó Ádám

Csapattagok

Kálmán Patrik

Papp Bálint

dr. Gróf László Mihály

Hája Gergő Zoltán

Dobronyi Benedek

Márk

XG5YQ1

HND2AH

EAS7U0

FEFSRB

IMQ1AR

kalmanpatrik1214@gmail.com

xypapbal@gmail.com

laszlo.m.grob@gmail.com

hejagergo@gmail.com

benedek.dobronyi@gmail.com

2026.05.09.

11. Grafikus felület specifikációja

A prototípus elkészültével és sikeres értékelésével az utolsó iterációhoz értünk: a meglévő, parancssorra (CLI) épülő rendszerhez illesztünk egy grafikus felhasználói felületet (GUI). Az alábbi tervek azt mutatják be, hogyan néz ki és hogyan működik a végleges Swing-alapú megjelenítés, milyen új és módosított osztályokat vezetünk be, és hogyan kapcsolódnak ezek az eddig is használt modell- és parancsréteghez. Az alapvető tervezési cél az volt, hogy a meglévő modell-osztályokat minél kevésbé kelljen módosítani: a GUI a prototípus parancsértelmezőjét hívja a vezérlés rétegen keresztül, a modell pedig egy egyszerű, push-alapú Observer interfészen át értesíti a nézetet a változásokról.

11.1 A grafikus interfész

A grafikus felület egy ablakos Swing-alkalmazás, amelynek központi eleme a pályatérkép. A jelölések – a feladatkiírásnak megfelelően – egyszerű színes téglalapok, ikonok és feliratok; a hangsúly a játék állapotának egyértelmű leolvasásán és a use-case-ek elérhetőségén van.

11.1.1 Főképernyők

A felület négy fő képernyőt használ:

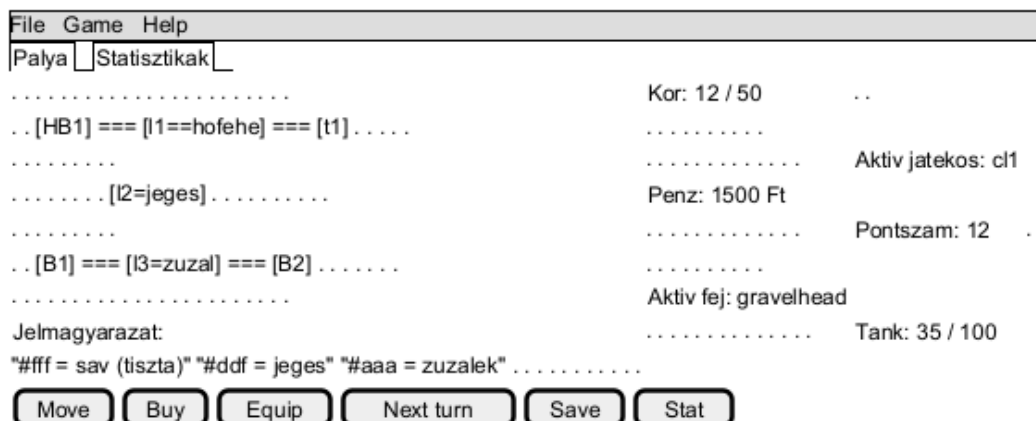
- Főmenü: új játék, pálya betöltése, mentés betöltése, beállítások, kilépés.
- Játékpálya nézet: középen a térkép, jobbra HUD, alul akció-toolbar, fent menüsor.
- Áruház (modal dialog): a HomeBase mezőre lépve a takarító játékos termékeket vásárolhat.
- Játék-vége képernyő: pontszámok, statisztikák, „Új játék” / „Mentés” / „Kilépés” gombok.

```

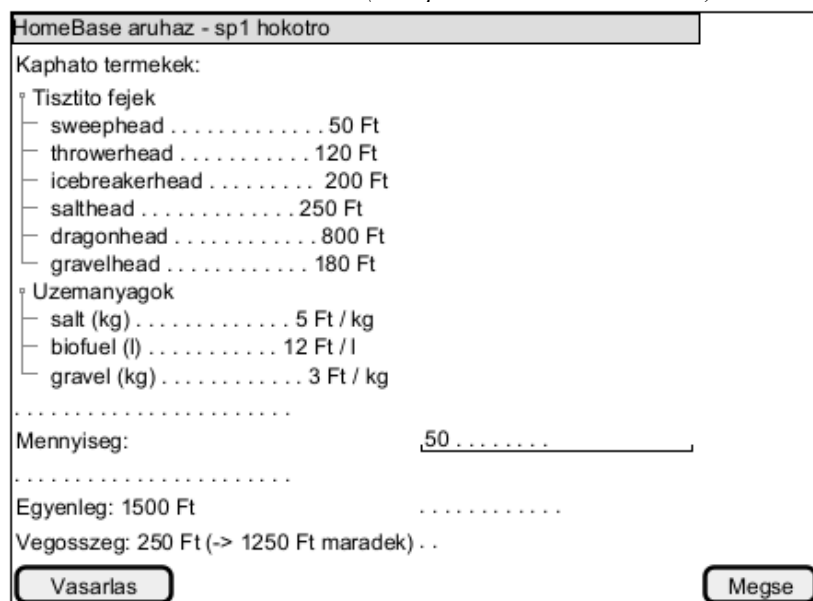
File Game Help
.....
.....
...                               GonoszOnosz - Hokotrok .....
.....
... [ Új játék ] .....
... [ Pálya betöltése ] .....
... [ Mentés betöltése ] .....
... [ Beállítások ] .....
... [ Kilépés ] .....
.....
.....
verzio: 11_proto.gui
konzulens: Marcsingo Adam

```

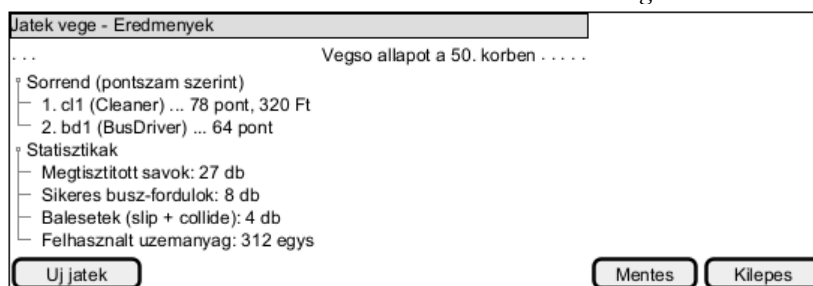
11.1. ábra – Főmenü mockup.



11.2. ábra – Játéknézet (térkép + HUD + akció-toolbar).



11.3. ábra – HomeBase áruház modal dialog.



11.4. ábra – Játék-vége eredménylista.

11.1.2 Pályaelemek és szereplők megjelenítése

- Lane (sáv): kitöltött téglalap; szürke alap, hó arányosan fehér réteg, jég kék overlay, zúzalék szürke pötty-mintás textúra, sóhatás zöldes glow.
- Road (út): a sávjait egy közös, halványoszürke konténer-keret fogja össze; a forward / backward sávok átellenes oldalon helyezkednek el.
- Building (általános épület): szürke téglalap az ID címkével.
- HomeBase: sárga ház-pictogrammal és „Áruház” jelvénnel; rákattintva nyílik a MarketDialog.
- Terminal: kék jelölővel és busz-pictogrammal; ide érkezve a busznak fordulója teljesül.

- Car: kék négyzet, fehér „C” betűvel; mozgáskor sárga keretet kap, megcsúszáskor pirosat.
- Bus: hosszúkás sárga téglalap, fekete „B” betűvel; végállomáson zöld keret jelzi a sikeres fordulót.
- SnowPlow: piros tank-forma, az aktív fej rövid kódjával (SW/TH/IB/SA/DR/GR).

11.1.3 Vezérlés (use-case lefedés)

A 7. heti dokumentum 7.2 fejezete sorolja fel a fő use-case-eket (pálya betöltése, mentés, determinisztikus mód, listázás, állapot, kör léptetése, busz mozgatása, hókotró mozgatása, vásárlás, fej csere, érvénytelen lépés, ...). Ezek mindegyike elérhető a GUI-ról:

- Menüsor / File: New / Load Map / Load Save / Save / Exit. — load, save, exit parancsok.
- Menüsor / Game: Next Turn (vagy SPACE billentyű), Toggle Random, Force Slip... — next_turn, random, force_slip parancsok.
- Menüsor / Help: About, Billentyűk listája.
- Egér / bal kattintás a térképen: kiválasztja a mezőt vagy a járművet (kék keret).
- Egér / bal kattintás kiválasztott jármű szomszédos mezőjén: move_bus / move_plow parancsot ad ki.
- Egér / jobb kattintás bármely objektumon: stat <id> – tooltipben mutatja az állapotot.
- Akció-toolbar / Move: a kiválasztott jármű mozgási módba lép, a célmezőre kattintás végzi a lépést.
- Akció-toolbar / Buy: HomeBase-en álló hókotrónál nyitja a MarketDialog-ot (buy parancsok).
- Akció-toolbar / Equip: HomeBase-en lecseréli az aktív fejet az attachments-ből (equip parancs).
- Akció-toolbar / Next turn: léptet egy kört.
- Akció-toolbar / Stat: a kiválasztott objektum [STATE] kimenete egy felugró ablakban.
- Akció-toolbar / Save: az aktuális memóriaképet egy fájlba menti (save).
- Billentyűk: SPACE = next_turn, ENTER = aktuális akció megerősítése, ESC = kiválasztás törlése, F1 = súgó, Ctrl+S = save, Ctrl+O = load.

Minden grafikus akció pontosan ugyanazt a parancsstringet építi fel és adja át a meglévő CommandParser-nek, mint amit a CLI-ben be lehet gépelni. Ebből következik, hogy a teszteléshez használt 20 db *_in.txt forgatókönyv a GUI-tól függetlenül továbbra is változatlan formában futtatható – a GUI csak egy alternatív input forrás.

11.2 A grafikus rendszer architektúrája

11.2.1 A felület működési elve

A felület MVC-szerű felosztásban működik:

- Modell: a prototípus már létező model.* osztályai (Lane, Vehicle és leszármazottai, Player és leszármazottai, GameLogic, Map, HomeBase, IntegratedMarket stb.).
- Nézet (új): a view.* csomag, amely tartalmazza a JFrame-et, a JPanel-eket és a modell-elemekhez tartozó FieldView / VehicleView leszármazottakat.
- Vezérlő (új): a controller.* csomag — az InputController és a CommandBridge. Az InputController az egér- és billentyűeseményeket fordítja le konkrét akciókra; a CommandBridge ezeket a már létező CommandParser parancsaivá alakítja.

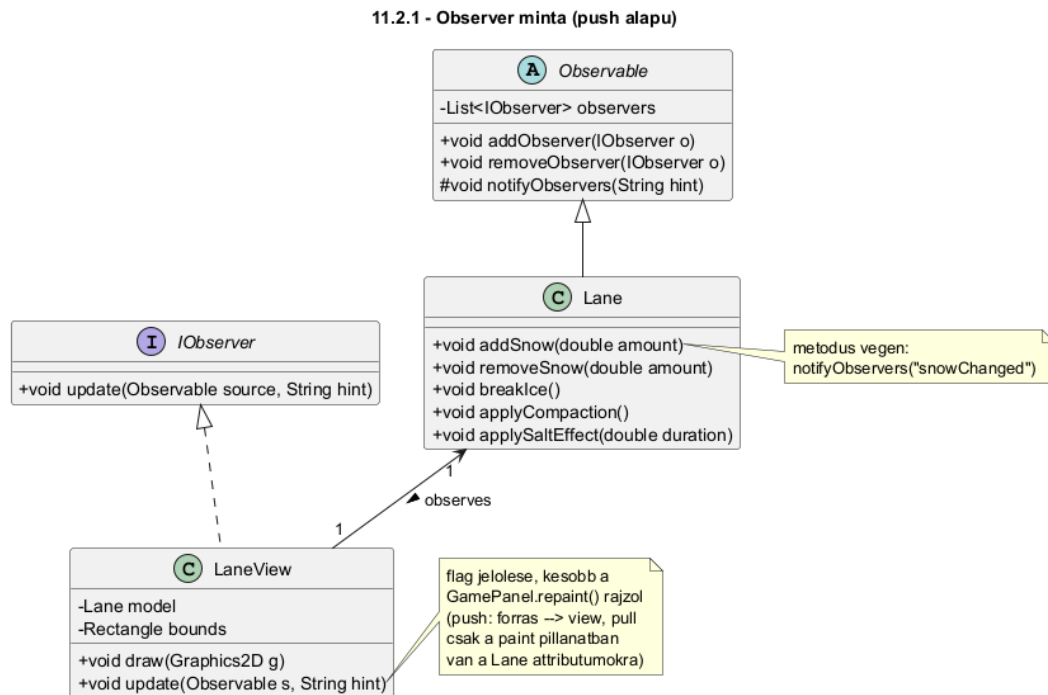
A modell és a nézet kapcsolatát push alapú Observer minta valósítja meg, egy kis pull jellegű kiegészítéssel:

- Push: minden állapotváltoztató modell-metódus (pl. Lane.addSnow, Vehicle.move, Player.addMoney) a végén meghív egy notifyObservers(hint) hívást. A view erre a

jelzésre csak egy „dirty” flag-et állít be magán, és kéri a befoglaló `GamePanel.repaint()`-jét.

- Pull (csak a paint pillanatában): a paint ciklusban a `FieldView/VehicleView` leszármazottak közvetlenül lekérdezik a modell aktuális attribútumait (`snowDepth`, `isFrozen`, `currentField` stb.) és ez alapján rajzolnak.

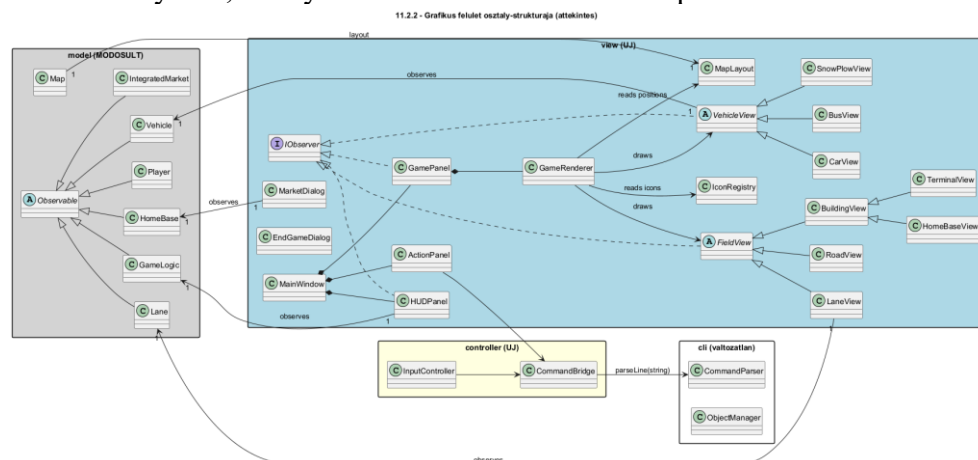
A modell soha nem hivatkozik konkrét view osztályra, és nincs benne `draw()` / `paint()` metódus. Az egyetlen GUI-orientált beavatkozás a `model.Observable` absztrakt osztály bevezetése, amelyet az érintett modell-osztályok ősosztályként vagy oldalági öröklődéssel kapnak meg, valamint a `notifyObservers(hint)` hívás beillesztése az állapotváltoztató metódusok végére.



11.5. ábra – Push-alapú Observer minta.

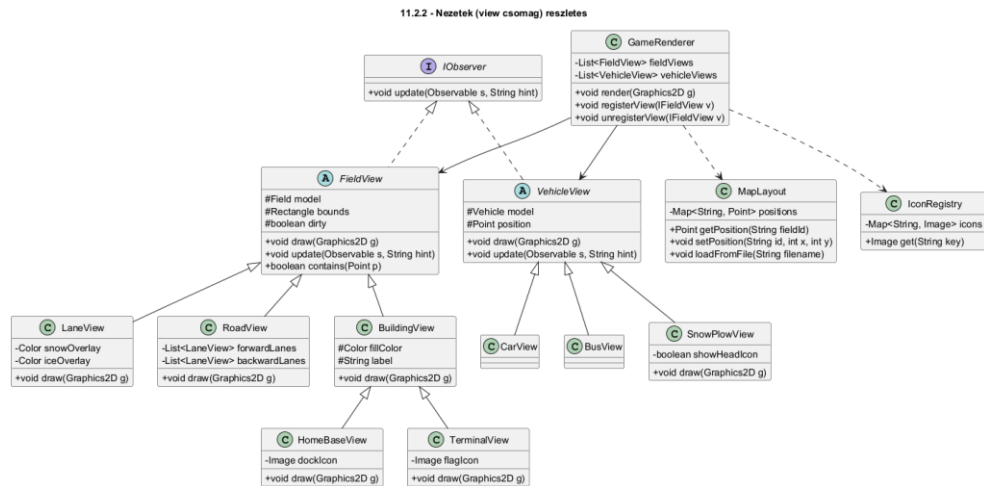
11.2.2 A felület osztály-struktúrája

Az alábbi áttekintő ábra mutatja az új csomagokat (view, controller) és azokat a meglévő modell- és cli-osztályokat, amelyekhez a GUI közvetlenül kapcsolódik.



11.6. ábra – Áttekintő osztálydiagram.

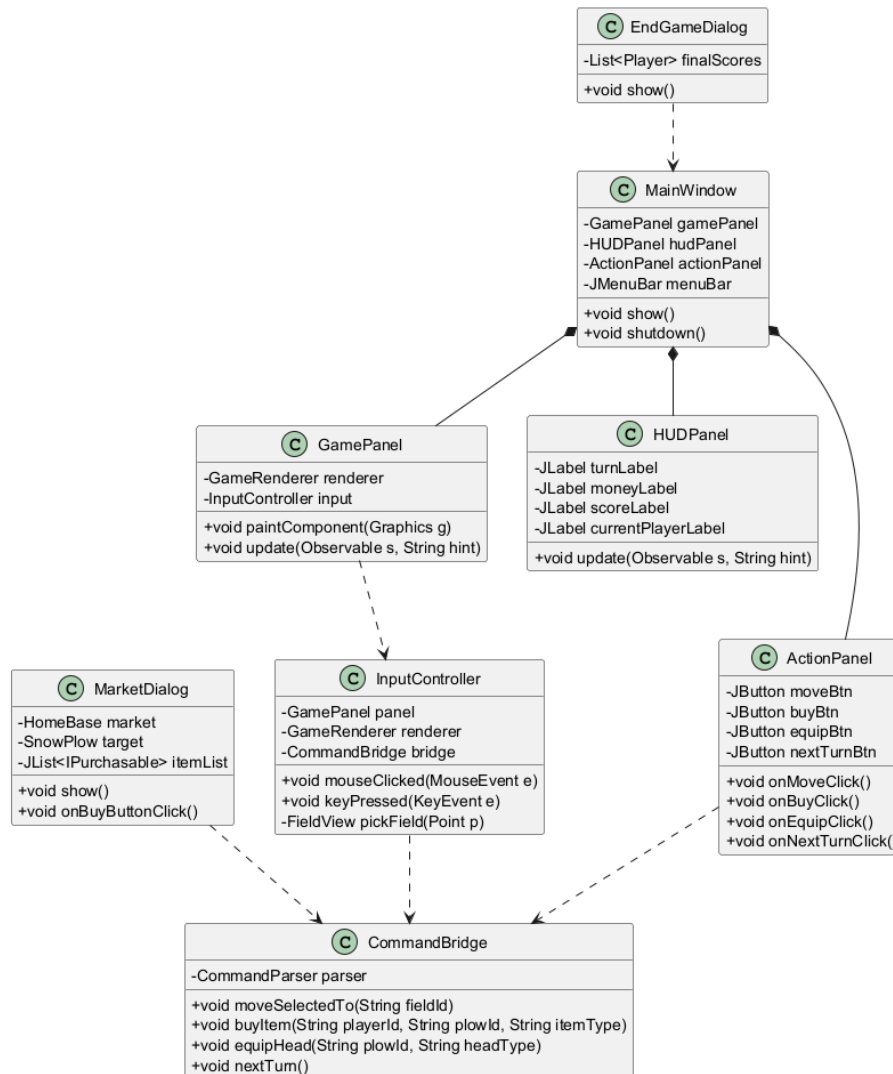
A nézet csomag belső struktúrája külön, részletes diagrammal:



11.7. ábra – view csomag részletes osztálydiagramja.

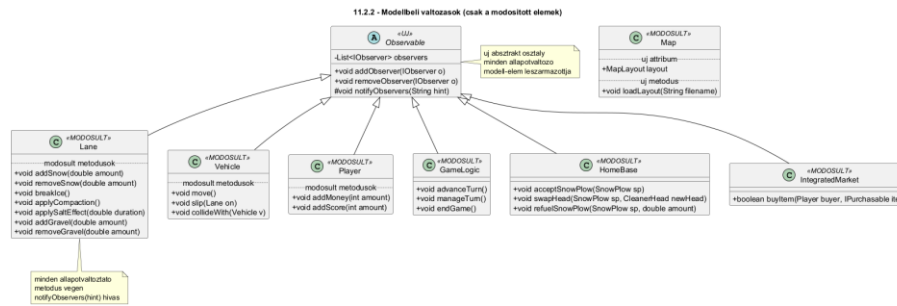
A főablak és a vezérlés komponensei:

11.2.2 - Foablak es vezerlo komponensek



11.8. ábra – MainWindow + Controller komponensek.

A modell oldalán bevezetett és érintett osztályok:



11.9. ábra – Modell-rétegbeli változások.

11.2.3 Új / kiegészítő pályafarmátum

A modell pozícióinformációt eddig nem tárolt – a CLI tesztek kizárólag az ID-alapú connect_fields gráffal dolgoznak. A grafikus felülethez két új, opcionális parancsot vezetünk be a meglévő konfigurációs nyelvbe (load és save kompatibilis):

- `set_position <field_id> <x> <y>`: egy mező képernyő-koordinátája pixelben, a layout-rácshoz viszonyítva.
- `set_road_orientation <road_id> <horizontal|vertical>`: egy út tengelyét rögzíti; ha hiányzik, a Road két végpontjából automatikusan kiszámítjuk.

A pozíció-információt a Map osztály új layout: MapLayout attribútuma tartja, és nem hat vissza a modell viselkedésére. Ha a betöltött *_in.txt nem tartalmaz set_position parancsokat, a MapLayout egy egyszerű automatikus elrendezést használ a Road-csatlakozások (connect_roads) alapján – így a meglévő 20 db tesztfájl változatlan formában futtatható marad.

11.3 A grafikus objektumok felsorolása

Ebben a fejezetben az új és a módosult osztályok kerülnek felsorolásra a 7-8. heti dokumentumokkal egyező formátumban. Régi, változatlan osztályokat nem ismételünk.

11.3.1 Új osztályok

1. Observable

- **Felelősség:** Az összes állapotváltozást közlő modell-osztály közös ősosztálya. Felelőssége az Observer-feliratkozottak nyilvántartása és értesítése. Nem tud arról, hogy a megfigyelői grafikus elemek; csak az IObservable interfészen keresztül kommunikál velük, így a modell teljesen elkülönül a megjelenítéstől.
- **Ősosztályok:** Nincs
- **Interfészek:** Nincs
- **Attribútumok:**
 - - observers: List<IObserver> – A feliratkozott megfigyelők listája.
- **Metódusok:**
 - + void addObserver(IObserver o): Felvesz egy új megfigyelőt.
 - **Algoritmus:**
 1. HA observers nem tartalmazza o-t, AKKOR az observers lista végéhez fűzi.
 - + void removeObserver(IObserver o): Kivesz egy megfigyelőt.
 - **Algoritmus:**
 1. Eltávolítja o-t az observers listából, ha szerepel benne.
 - # void notifyObservers(String hint): Sorra értesíti a megfigyelőket; a hint rövid jelzés a változás típusáról.
 - **Algoritmus:**

1. CIKLUS minden $o \in \text{observers}$ elemre: meghívja `o.update(this, hint)`.

2. IObserver

- **Felelősség:** Közös interfész minden olyan grafikus osztálynak, amely modell-elemek változásairól kíván értesülni. Egyetlen metódusa van; a megvalósító view tárolja a forrás-objektum referenciáját, és lekérdezi belőle, ami a frissítéshez kell.
- **Ősosztályok:** Nincs
- **Interfészek:** Nincs
- **Attribútumok:**
 - Nincs.
- **Metódusok:**
 - + void `update(Observable source, String hint)`: Az `Observable.notifyObservers` hívja minden feliratkozottra.
 - **Algoritmus:**
 1. Általános séma: dirty flag beállítása + a befoglaló `GamePanel.repaint()` kérése.

3. MainWindow

- **Felelősség:** A program főablaka (`JFrame`). Felelős a felső menüsor, a központi `GamePanel`, a jobb oldali `HUDPanel` és az alsó `ActionPanel` elhelyezéséért, az ablak életciklusáért és a globális billentyű-leképezésekért.
- **Ősosztályok:** `javax.swing.JFrame`
- **Interfészek:** Nincs
- **Attribútumok:**
 - - `gamePanel`: `GamePanel` – A pálya rajzfelülete.
 - - `hudPanel`: `HUDPanel` – Jobb oldali információs panel.
 - - `actionPanel`: `ActionPanel` – Alsó akció-toolbar.
 - - `menuBar`: `JMenuBar` – Felső menüsor (File / Game / Help).
- **Metódusok:**
 - + void `show()`: Megjeleníti az ablakot, fókuszálja a `GamePanel`-t.
 - **Algoritmus:**
 1. `BorderLayout` szerint hozzáadja a menüsort, `gamePanel`-t (`CENTER`), `hudPanel`-t (`EAST`), `actionPanel`-t (`SOUTH`).
 2. `setVisible(true)` az ablakra; `gamePanel.requestFocus()`.
 - + void `shutdown()`: Erőforrások felszabadítása, ablak bezárása.
 - **Algoritmus:**
 1. Bezárja a megnyitott dialógusokat.
 2. Lefuttatja a parser exit parancsát.
 3. `dispose()` az ablakra.
 - - void `buildMenuBar()`: Felépíti a menüsort és bekötögeti az `ActionListener`-eket.
 - **Algoritmus:**
 1. File menü: New, Load Map, Load Save, Save, Exit.
 2. Game menü: Next Turn, Toggle Random, Force Slip...
 3. Help menü: About, Billentyűk listája.
 4. Minden gomb `ActionListener`-e a `CommandBridge` megfelelő metódusát hívja.

4. GamePanel

- **Felelősség:** A térkép rajzfelülete. Felelőssége: a paintComponent ciklusban megrendelni a GameRenderer-től a teljes pálya kirajzolását; az egér- és billentyűeseményeket továbbítani az InputController-nek. Maga is IObservable a GameLogic-ra.
- **Össztályok:** javax.swing.JPanel
- **Interfészek:** IObservable
- **Attribútumok:**
 - - renderer: GameRenderer – A renderelő motor.
 - - input: InputController – Egér/billentyű kezelő.
- **Metódusok:**
 - + void paintComponent(Graphics g): Felülírt Swing metódus, a teljes térképet kirajzolja.
 - **Algoritmus:**
 1. super.paintComponent(g) hívása.
 2. renderer.render(g) hívás a teljes pálya kirajzolásához.
 - + void update(Observable source, String hint): A megfigyelt modell-elem változásakor újrarajzol.
 - **Algoritmus:**
 1. this.repaint() hívása.

5. HUDPanel

- **Felelősség:** A jobb oldalt elhelyezkedő státusz-panel: aktuális kör, aktív játékos, pénz, pontszám, aktív hókotró-fej és tank-állapot. Megfigyelője a GameLogic-nak és a Player-eknek.
- **Össztályok:** javax.swing.JPanel
- **Interfészek:** IObservable
- **Attribútumok:**
 - - turnLabel: JLabel – „Kör: X / Y” felirat.
 - - currentPlayerLabel: JLabel – Aktív játékos azonosítója.
 - - moneyLabel: JLabel – Aktív takarító játékos pénze.
 - - scoreLabel: JLabel – Aktív játékos pontszáma.
 - - headLabel: JLabel – Aktív SnowPlow fejtípusa és tankja.
- **Metódusok:**
 - + void update(Observable source, String hint): A jelzésnek megfelelő JLabel-eket frissíti.
 - **Algoritmus:**
 1. HA hint = „turnAdvanced”, AKKOR turnLabel frissítése.
 2. HA hint = „moneyChanged”, AKKOR moneyLabel frissítése.
 3. HA hint = „scoreChanged”, AKKOR scoreLabel frissítése.
 4. HA hint = „headSwapped” VAGY „refuel”, AKKOR headLabel frissítése.

6. ActionPanel

- **Felelősség:** Az ablak alján elhelyezett akció-gombsor: Move, Buy, Equip, Next turn, Save, Stat. A gombokra kötött ActionListener-ek a CommandBridge megfelelő metódusát hívják.
- **Össztályok:** javax.swing.JPanel
- **Interfészek:** Nincs
- **Attribútumok:**
 - - moveBtn: JButton – Lépés mód aktiválása.

- - buyBtn: JButton – Áruház megnyitása.
- - equipBtn: JButton – Fejcsere a HomeBase-en.
- - nextTurnBtn: JButton – next_turn parancs kiadása.
- - saveBtn: JButton – save parancs.
- - statBtn: JButton – stat parancs a kijelölt objektumra.
- **Metódusok:**
 - + void onMoveClick(): Az InputController belép „mozgás-módba”.
 - **Algoritmus:**
 1. input.setActionMode(MOVE).
 2. A következő mező-kattintás mozgási parancsot generál a kiválasztott járművel.
 - + void onBuyClick(): MarketDialog megnyitása ha SnowPlow HomeBase-en áll.
 - **Algoritmus:**
 1. HA selected SnowPlow ÉS selected.currentField HomeBase, AKKOR új MarketDialog.show().
 2. KÜLÖNBEN: hibajelzés.
 - + void onEquipClick(): equip parancsot ad ki HomeBase-en.
 - **Algoritmus:**
 1. HA selected SnowPlow ÉS HomeBase-en áll ÉS van választott head, AKKOR bridge.equipHead(plowId, headType).
 - + void onNextTurnClick(): next_turn parancsot ad ki.
 - **Algoritmus:**
 1. bridge.nextTurn() hívása.
 - + void onSaveClick(): save parancsot futtat fájlba.
 - **Algoritmus:**
 1. JFileChooser megnyitása.
 2. HA a felhasználó választott fájlt, bridge.save(path) hívása.
 - + void onStatClick(): stat parancsot futtat a kijelölt objektumra.
 - **Algoritmus:**
 1. HA van kijelölt objektum, bridge.stat(id) hívás, az eredményt egy JOptionPane-ben mutatja.

7. MarketDialog

- **Felelősség:** Modal dialog ablak az IntegratedMarket bemutatására és a tranzakciók kiváltására. Megfigyelője a takarító Player-nek (pénzváltozás) és az IntegratedMarket-nek.
- **Össztályok:** javax.swing.JDialog
- **Interfészek:** IObserver
- **Attribútumok:**
 - - market: IntegratedMarket – A megfigyelt áruház.
 - - target: SnowPlow – A vásárlás célja.
 - - buyer: Cleaner – A vásárló játékos.
 - - itemList: JList<IPurchasable> – Termék-választó lista.
 - - qtyField: JTextField – Üzemanyag-mennyiség.
 - - balanceLbl: JLabel – Aktuális egyenleg.
- **Metódusok:**
 - + void show(): Megjeleníti a dialógust modal módban.
 - **Algoritmus:**
 1. itemList feltöltése market.getAvailableItems() alapján.

- 2. balanceLbl feltöltése buyer.money értékkel.
 - 3. setVisible(true).
- + void onBuyButtonClick(): Felépíti a buy parancsstringet.
 - **Algoritmus:**
 - 1. HA itemList.selected üzemanyag, AKKOR amount = qtyField; bridge.buyItem(buyerId, plowId, type, amount).
 - 2. KÜLÖNBEN (fej): bridge.buyItem(buyerId, plowId, headType, 0).
- + void update(Observable source, String hint): Pénz / lista változásakor frissít.
 - **Algoritmus:**
 - 1. HA hint = „moneyChanged”, AKKOR balanceLbl új szöveggel.
 - 2. HA hint = „itemBought”, AKKOR itemList újratöltése.

8. EndGameDialog

- **Felelősség:** A játék végén megjelenő modal dialog: játékosok rangsorolt listája pontszám és pénz szerint, valamint statisztikák.
- **Ősosztályok:** javax.swing.JDialog
- **Interfészek:** Nincs
- **Attribútumok:**
 - - finalScores: List<Player> – A játékosok rangsora.
 - - stats: Map<String,Integer> – Aggregált statisztikák.
- **Metódusok:**
 - + void show(): Megjeleníti a dialógust modal módban.
 - **Algoritmus:**
 - 1. A finalScores rendezése pontszám szerint csökkenőre.
 - 2. Eredménytábla felépítése.
 - 3. setVisible(true).

9. GameRenderer

- **Felelősség:** A teljes térkép rajzolásának orchestratora. Egy listán tartja az összes regisztrált FieldView és VehicleView példányt.
- **Ősosztályok:** Nincs
- **Interfészek:** Nincs
- **Attribútumok:**
 - - fieldViews: List<FieldView> – Pálya-elem nézetek.
 - - vehicleViews: List<VehicleView> – Jármű-nézetek.
 - - layout: MapLayout – Pozíció-táblázat.
 - - icons: IconRegistry – Ikonok cache-e.
- **Metódusok:**
 - + void render(Graphics2D g): Sorra rajzolja a regisztrált nézeteket.
 - **Algoritmus:**
 - 1. Először az utak (RoadView) kerülnek kirajzolásra.
 - 2. Aztán a sávok (LaneView).
 - 3. Aztán az épületek (BuildingView és leszármazottai).
 - 4. Végül a járművek (VehicleView), így a mozgó elemek a térkép tetején látszanak.
 - + void registerView(IFieldView v): View regisztrálása load után.
 - **Algoritmus:**
 - 1. A v típusától függően a megfelelő listához (fieldViews / vehicleViews) hozzáfűzi v-t.

- + void unregisterView(IFieldView v): View eltávolítása.
 - **Algoritmus:**
 1. v eltávolítása a megfelelő listából.

10. FieldView (abstract)

- **Felelősség:** Pálya-elem nézetek közös őssztálya. A geometria és a megfigyelés közös részét tartalmazza.
- **Őssztályok:** Nincs
- **Interfészek:** IObservable
- **Attribútumok:**
 - # model: Field – A megfigyelt modell-elem.
 - # bounds: Rectangle – A térképen elfoglalt pixel-téglalap.
 - # dirty: boolean – Push-jelzés óta volt-e változás.
- **Metódusok:**
 - + void draw(Graphics2D g): Absztrakt; a leszármazott rajzolja ki magát.
 - **Algoritmus:**
 1. Algoritmus: absztrakt metódus, nincs konkrét implementáció.
 - + void update(Observable source, String hint): Beállítja a dirty flag-et és kéri a GamePanel.repaint()-jét.
 - **Algoritmus:**
 1. dirty = true.
 2. Az ősbeli GamePanel.repaint() közvetett hívása.
 - + boolean contains(Point p): Egér-koordináta tesztelés.
 - **Algoritmus:**
 1. VISSZATÉRÉSI ÉRTÉK bounds.contains(p).

11. LaneView

- **Felelősség:** Egy Lane (forgalmi sáv) grafikus megjelenítője. A sáv aktuális snowDepth-jét, isFrozen-t, gravelDepth-et, saltEffect-et és isBlocked-et különböző overlay-ekkel ábrázolja.
- **Őssztályok:** FieldView
- **Interfészek:** IObservable
- **Attribútumok:**
 - Nincs.
- **Metódusok:**
 - + void draw(Graphics2D g): Lane attribútumok lekérdezése + overlay-ek.
 - **Algoritmus:**
 1. Háttér téglalap kirajzolása szürke alapszínnel.
 2. HA snowDepth > 0, AKKOR fehér overlay arányosan a snowDepth értékével.
 3. HA isFrozen igaz, AKKOR kék overlay.
 4. HA gravelDepth > 0, AKKOR szürke pötty-mintás textúra.
 5. HA saltEffect > 0, AKKOR zöldes glow.
 6. HA isBlocked() igaz, AKKOR piros átlós csíkok.
 7. ID címke a téglalap közepére.

12. RoadView

- **Felelősség:** Egy Road konténer-keretét rajzolja, amely körbeöleli a hozzá tartozó forwardLanes és backwardLanes sávokat.
- **Őssztályok:** FieldView

- **Interfészek:** IObservable
- **Attribútumok:**
 - - forwardLanes: List<LaneView> – Saját forward irány LaneView-i.
 - - backwardLanes: List<LaneView> – Saját backward irány LaneView-i.
- **Metódusok:**
 - + void draw(Graphics2D g): Halványszürke befoglaló keret.
 - **Algoritmus:**
 1. A forward+backward LaneView-k uniója kiszámítása.
 2. Befoglaló téglalap halványszürke kerettel.
 3. Az út ID címke kiírása.

13. BuildingView

- **Felelősség:** Általános épület-nézet közös őosztálya. A leszármazottak (HomeBaseView, TerminalView) a fillColor és az ikon kódját változtatják meg.
- **Őosztályok:** FieldView
- **Interfészek:** IObservable
- **Attribútumok:**
 - # fillColor: Color – A téglalap kitöltő színe.
 - # label: String – A megjelenített címke (általában az ID).
- **Metódusok:**
 - + void draw(Graphics2D g): Kitöltött téglalap a fillColor-rel, fekete kerettel, középre igazítva a label.
 - **Algoritmus:**
 1. g.setColor(fillColor); g.fillRect(bounds).
 2. g.setColor(BLACK); g.drawRect(bounds).
 3. g.drawString(label, középpont).

14. HomeBaseView

- **Felelősség:** A HomeBase épülethez tartozó vizuális nézet: sárga ház-pictogram és „Áruház” jelvény.
- **Őosztályok:** BuildingView
- **Interfészek:** IObservable
- **Attribútumok:**
 - - dockIcon: Image – A telephelyet jelölő pictogram.
- **Metódusok:**
 - + void draw(Graphics2D g): Sárga ház + dockIcon + áruház jelvény.
 - **Algoritmus:**
 1. super.draw(g) (sárga téglalap).
 2. g.drawImage(dockIcon, bounds.x+pad, bounds.y+pad).
 3. HA market nem üres, áruház jelvény kirajzolása.

15. TerminalView

- **Felelősség:** Egy buszvégállomás (Terminal) nézete: kék jelölő és busz-pictogram. Sikeres busz-érkezéskor zöld keret villan rá.
- **Őosztályok:** BuildingView
- **Interfészek:** IObservable
- **Attribútumok:**
 - - flagIcon: Image – Buszvégállomás-jelölő ikon.
 - - highlightTtl: int – Hány frame-ig villog a sikeres érkezés zöld kerete.
- **Metódusok:**

- + void draw(Graphics2D g): Kék téglalap + busz-pictogram + zöld keret ha highlight.
 - **Algoritmus:**
 1. super.draw(g).
 2. g.drawImage(flagIcon, ...).
 3. HA highlightTtl > 0, AKKOR zöld keret kirajzolása + highlightTtl csökkentése.
- + void update(Observable source, String hint): „arrived” hint-re highlightTtl-t beállítja.
 - **Algoritmus:**
 1. HA hint = „arrived”, AKKOR highlightTtl = 8.
 2. Ősbeli FieldView.update hívása (dirty + repaint).

16. VehicleView (abstract)

- **Felelősség:** Jármű-nézetek közös őssztálya: a járművet az aktuális Lane / Building pozícióra rajzolja.
- **Őssztályok:** Nincs
- **Interfészek:** IObserver
- **Attribútumok:**
 - # model: Vehicle – A megfigyelt jármű.
 - # position: Point – Az aktuális pixel-koordináta.
- **Metódusok:**
 - + void draw(Graphics2D g): Absztrakt; a leszármazott rajzol.
 - **Algoritmus:**
 1. Algoritmus: absztrakt metódus, nincs konkrét implementáció.
 - + void update(Observable source, String hint): „moved” hint-re újraszámítja a position-t.
 - **Algoritmus:**
 1. HA hint = „moved”, AKKOR position = MapLayout.getPosition(model.currentField.id).
 2. this.dirty = true; GamePanel.repaint().

17. CarView

- **Felelősség:** Egy NPC autó (Car) megjelenítője. Kék négyzet, fehér „C” betűvel; megcsúszáskor piros, ütközéskor sárga keretes pulzáló jelzés.
- **Őssztályok:** VehicleView
- **Interfészek:** IObserver
- **Attribútumok:**
 - Nincs.
- **Metódusok:**
 - + void draw(Graphics2D g): Kék négyzet + C betű + státuszkeret.
 - **Algoritmus:**
 1. g.setColor(BLUE); g.fillRect(position).
 2. g.drawString(„C”, ...).
 3. HA model.waitTurns > 0, AKKOR fakó tónus.
 4. HA legutóbbi hint slip volt, AKKOR piros keret; ha collide, AKKOR sárga keret.

18. BusView

- **Felelősség:** A busz nézete: hosszúkas sárga téglalap, fekete „B” betűvel.

- **Ősosztályok:** VehicleView
- **Interfészek:** IObservable
- **Attribútumok:**
 - Nincs.
- **Metódusok:**
 - + void draw(Graphics2D g): Sárga téglalap + B betű + státusz overlay.
 - **Algoritmus:**
 1. g.setColor(YELLOW); g.fillRect(position, hosszú/keskeny).
 2. g.drawString(„B”, ...).
 3. HA model.isFunctioning hamis, AKKOR fakó ábrázolás + disabledTurnsLeft kis számmal.

19. SnowPlowView

- **Felelősség:** A hókotró nézete: piros tank-forma, az aktív fej rövid kódjával (SW/TH/IB/SA/DR/GR). Tank-állapot kis sávban a járművön belül.
- **Ősosztályok:** VehicleView
- **Interfészek:** IObservable
- **Attribútumok:**
 - - showHeadIcon: boolean – Az aktív fej rövidítését rajzolja-e.
- **Metódusok:**
 - + void draw(Graphics2D g): Piros téglalap + fej rövidítés + tank csík.
 - **Algoritmus:**
 1. g.setColor(RED); g.fillRect(position).
 2. HA showHeadIcon, AKKOR a fej kétbetűs rövidítését felülre.
 3. HA model.activeHead.usesFuel, AKKOR kis tank-csík fuelAmount alapján.

20. MapLayout

- **Felelősség:** A pálya elemeinek képernyő-koordinátáit tárolja. A modell-logika nem támaszkodik rá.
- **Ősosztályok:** Nincs
- **Interfészek:** Nincs
- **Attribútumok:**
 - - positions: Map<String,Point> – field_id → pixel pozíció.
 - - roadOrientation: Map<String,String> – road_id → „horizontal”/„vertical”.
- **Metódusok:**
 - + Point getPosition(String fieldId): Pozíció lekérdezés.
 - **Algoritmus:**
 1. HA positions tartalmazza fieldId-t, VISSZATÉRÉSI ÉRTÉK positions.get(fieldId).
 2. KÜLÖNBEN: autoLayout-ból számított érték.
 - + void setPosition(String id, int x, int y): Pozíció beállítása.
 - **Algoritmus:**
 1. positions.put(id, new Point(x, y)).
 - + void loadFromFile(String filename): Külön layout fájl betöltése.
 - **Algoritmus:**
 1. A fájl minden sora egy set_position parancs.
 2. Soronként parsolva setPosition hívás.
 - + void autoLayout(Map gameMap): Egyszerű automatikus elrendezés.
 - **Algoritmus:**

1. A connect_roads gráfból egy lineáris útvonal-rajzot generál.
2. Minden Road forwardLanes / backwardLanes egymás mellé.
3. Az épületek a Road-jaik mellett.
4. A pozíciókat positions-be teszi.

21. IconRegistry

- **Felelősség:** Az ikonok és sprite-ok cache-elője.
- **Ősosztályok:** Nincs
- **Interfészek:** Nincs
- **Attribútumok:**
 - - icons: Map<String,Image> – név → kép.
- **Metódusok:**
 - + Image get(String key): Az adott nevű ikont visszaadja.
 - **Algoritmus:**
 1. HA icons tartalmazza key-t, VISSZATÉRÉSI ÉRTÉK icons.get(key).
 2. KÜLÖNBEN: betölti resources/icons/<key>.png-t.
 3. Ha az se elérhető, fallback színes téglalap kép.

22. InputController

- **Felelősség:** A GamePanel egér- és billentyűeseményeinek kezelője. Felelőssége: a kattintási koordináta alapján kiválasztani a fókuszált pálya-objektumot, az aktuális akció-módot figyelembe venni, majd a parancsot átadni a CommandBridge-nek.
- **Ősosztályok:** Nincs
- **Interfészek:** MouseListener, KeyListener
- **Attribútumok:**
 - - panel: GamePanel – A figyelt rajzfelület.
 - - renderer: GameRenderer – A regisztrált nézetek olvasásához.
 - - bridge: CommandBridge – A parancsstringek átadásához.
 - - selected: Object – A jelenleg kiválasztott objektum.
 - - actionMode: ActionMode – Aktuális akció-mód (NONE / MOVE / ...).
- **Metódusok:**
 - + void mouseClicked(MouseEvent e): Egér-kattintás kezelése.
 - **Algoritmus:**
 1. HA actionMode = MOVE ÉS a kiválasztott jármű szomszédos mezőjére kattintottunk: bridge.moveSelectedTo(...).
 2. KÜLÖNBEN bal kattintás: kiválasztja a kattintott objektumot (pickField vagy pickVehicle).
 3. Jobb kattintás: bridge.stat(id) a kattintott objektumra.
 - + void keyPressed(KeyEvent e): Billentyű-kezelő.
 - **Algoritmus:**
 1. SPACE: bridge.nextTurn().
 2. ESC: selected = null, actionMode = NONE.
 3. Nyilak: az aktuális járművet a megfelelő irány első szomszédjára lépteti.
 4. Ctrl+S / Ctrl+O: save / load file-chooser.
 - - FieldView pickField(Point p): Kattintási pont alapján FieldView-t keres.
 - **Algoritmus:**
 1. A renderer fieldViews listáján visszafelé iterálva (felülről lefelé).

2. Az első FieldView, amelyre `v.contains(p)` igaz, a találat.
- - `VehicleView pickVehicle(Point p)`: Ugyanez járművekre.
 - **Algoritmus:**
 1. A `renderer vehicleViews` listáján visszafelé iteráció.
 2. Az első olyan view, amelyik `bounds`-ja tartalmazza `p`-t.

23. CommandBridge

- **Felelősség:** A GUI és a CLI közötti fordító. A hívók egyszerű metódusokat hívnak; ez az osztály felépíti a megfelelő szöveges parancsot és átadja a `CommandParser.parseLine`-nak.
- **Össztályok:** Nincs
- **Interfészek:** Nincs
- **Attribútumok:**
 - - `parser`: `CommandParser` – A meglévő parancsértelmező.
- **Metódusok:**
 - + `void moveSelectedTo(String fromVehicleId, String toFieldId, boolean isPlow)`: Move parancs felépítése.
 - **Algoritmus:**
 1. HA `isPlow`, AKKOR `parser.parseLine(„move_plow „ + fromVehicleId + „ „ + toFieldId)`.
 2. KÜLÖNBEN `parser.parseLine(„move_bus „ + fromVehicleId + „ „ + toFieldId)`.
 - + `void buyItem(String playerId, String plowId, String itemType, int amount)`: Buy parancs felépítése.
 - **Algoritmus:**
 1. HA `amount > 0` (üzemanyag): `parser.parseLine(„buy „ + playerId + „ „ + plowId + „ „ + itemType + „ „ + amount)`.
 2. KÜLÖNBEN (fej): `parser.parseLine(„buy „ + playerId + „ „ + plowId + „ „ + itemType)`.
 - + `void equipHead(String plowId, String headType)`: Equip parancs.
 - **Algoritmus:**
 1. `parser.parseLine(„equip „ + plowId + „ „ + headType)`.
 - + `void nextTurn()`: Next turn parancs.
 - **Algoritmus:**
 1. `parser.parseLine(„next_turn“)`.
 - + `void load(String filename)`: Load parancs.
 - **Algoritmus:**
 1. `parser.parseLine(„load „ + filename)`.
 - + `void save(String filename)`: Save parancs.
 - **Algoritmus:**
 1. `parser.parseLine(„save „ + filename)`.
 - + `String stat(String objectId)`: Stat parancs eredményének visszaadása.
 - **Algoritmus:**
 1. A standard kimenet ideiglenes átirányítása.
 2. `parser.parseLine(„stat „ + objectId)`.
 3. A capturált szöveg visszaadása.
 - + `void forceSlip(String vehicleId, boolean value)`: Force slip parancs.
 - **Algoritmus:**
 1. `parser.parseLine(„force_slip „ + vehicleId + „ „ + value)`.

24. ViewBinder

- **Felelősség:** Egy load végén lefutó segédobjektum: minden modell-példányra létrehozza a megfelelő nézetet, beköti az Observer kapcsolatot, regisztrálja a GameRenderer-be.
- **Ősosztályok:** Nincs
- **Interfészek:** Nincs
- **Attribútumok:**
 - - registered: List<IObserver> – A jelenleg felvett nézetek (későbbi unbindhoz).
- **Metódusok:**
 - + void bindAll(ObjectManager om, GameRenderer renderer): Új load végén nézeteket rendel a teljes pálya-térképhez.
 - **Algoritmus:**
 1. unbindAll(renderer) hívása először.
 2. CIKLUS minden obj ∈ om.getAll() elemre:
 3. HA obj Lane: új LaneView; obj.addObserver(view); renderer.registerView(view).
 4. HA obj Road: hasonlóan RoadView.
 5. HA obj Building: BuildingView vagy a megfelelő leszármazott.
 6. HA obj Vehicle: VehicleView a megfelelő alfajjal.
 7. HA obj Player: a HUD-hoz kötés (player.addObserver(hud)).
 - + void unbindAll(GameRenderer renderer): Az új load előtt minden korábbi nézetet leiratkoztat és eltávolít.
 - **Algoritmus:**
 1. CIKLUS minden v ∈ registered elemre: v forrásmodelljéből eltávolítja a megfigyelőt.
 2. renderer kiürítése.
 3. registered.clear().

25. MainApp

- **Felelősség:** A grafikus alkalmazás belépési pontja, a meglévő ProtoApp helyett (GUI módban). A ProtoApp változatlanul megmarad a CLI-tesztekhez.
- **Ősosztályok:** Nincs
- **Interfészek:** Nincs
- **Attribútumok:**
 - Nincs.
- **Metódusok:**
 - + static void main(String[] args): GUI mód indítása.
 - **Algoritmus:**
 - Példányosít egy új ObjectManager-t.
 - Példányosít egy új GameLogic-ot.
 - Példányosít egy új CommandParser-t és beregisztrálja az összes parancsot (mint a ProtoApp).
 - Példányosít egy új GameRenderer, InputController, MainWindow objektumot.
 - SwingUtilities.invokeLater() -> mainWindow.show()).

11.3.2 Megváltozott osztályok (csak a változások)

26. Lane – módosult

- **Felelősség:** Felelőssége változatlan; az új feladat csak a notifyObservers-hívások beillesztése a state-változó metódusok végére.
- **Őszosztályok:** Observable (új ős)
- **Interfészek:** Field (változatlan)
- **Attribútumok:**
 - Nincs.
- **Metódusok:**
 - + void addSnow(double amount): Eredeti + notifyObservers.
 - **Algoritmus:**
 1. Eredeti algoritmus végrehajtása.
 2. notifyObservers(„snowChanged”).
 - + void removeSnow(double amount): Eredeti + notifyObservers.
 - **Algoritmus:**
 1. Eredeti algoritmus.
 2. notifyObservers(„snowChanged”).
 - + void breakIce(): Eredeti + notifyObservers.
 - **Algoritmus:**
 1. Eredeti algoritmus.
 2. notifyObservers(„iceChanged”).
 - + void removeIce(): Eredeti + notifyObservers.
 - **Algoritmus:**
 1. Eredeti algoritmus.
 2. notifyObservers(„iceChanged”).
 - + void applyCompaction(): Eredeti + notifyObservers ha checkFreeze átállította.
 - **Algoritmus:**
 1. Eredeti algoritmus végrehajtása.
 2. HA isFrozen most lett igaz, AKKOR notifyObservers(„iceChanged”).
 - + void applySaltEffect(double duration): Eredeti + notifyObservers.
 - **Algoritmus:**
 1. Eredeti algoritmus.
 2. notifyObservers(„saltChanged”).
 - + void addGravel(double amount): Eredeti + notifyObservers.
 - **Algoritmus:**
 1. Eredeti algoritmus.
 2. notifyObservers(„gravelChanged”).
 - + void removeGravel(double amount): Eredeti + notifyObservers.
 - **Algoritmus:**
 1. Eredeti algoritmus.
 2. notifyObservers(„gravelChanged”).
 - + void accept(Vehicle v): Eredeti + notifyObservers.
 - **Algoritmus:**
 1. Eredeti algoritmus (csúszás-ellenőrzéssel együtt).
 2. notifyObservers(„vehiclesChanged”).
 - + void remove(Vehicle v): Eredeti + notifyObservers.
 - **Algoritmus:**
 1. Eredeti algoritmus.
 2. notifyObservers(„vehiclesChanged”).

27. Vehicle – módosult

- **Felelősség:** Az absztrakt jármű-osztály; leszármazottak (Car, Bus, SnowPlow) saját move/slip/collide implementációjuk végén notifyObservers-t hívnak.
- **Ősosztályok:** Observable (új ős)
- **Interfészek:** Nincs
- **Attribútumok:**
 - Nincs.
- **Metódusok:**
 - + void move(): Leszármazottak felüldefiniálják.
 - **Algoritmus:**
 1. Eredeti algoritmus a leszármazott szerint.
 2. notifyObservers(„moved”).
 - + void slip(Lane on): Leszármazottak felüldefiniálják.
 - **Algoritmus:**
 1. Eredeti algoritmus a leszármazott szerint.
 2. notifyObservers(„slipped”).
 - + void collideWith(Vehicle v): Leszármazottak felüldefiniálják.
 - **Algoritmus:**
 1. Eredeti algoritmus.
 2. notifyObservers(„collided”).

28. Player – módosult

- **Felelősség:** A pénzhez és pontszámhoz tartozó setterek/inkrementálók notifyObservers-t hívnak.
- **Ősosztályok:** Observable (új ős)
- **Interfészek:** Nincs
- **Attribútumok:**
 - Nincs.
- **Metódusok:**
 - + void addMoney(int amount): Eredeti + notifyObservers.
 - **Algoritmus:**
 1. money += amount.
 2. notifyObservers(„moneyChanged”).
 - + void addScore(int amount): Eredeti + notifyObservers.
 - **Algoritmus:**
 1. score += amount.
 2. notifyObservers(„scoreChanged”).

29. GameLogic – módosult

- **Felelősség:** A körléptetés és a játék-vége detekció után notifyObservers-t hív.
- **Ősosztályok:** Observable (új ős)
- **Interfészek:** Nincs
- **Attribútumok:**
 - Nincs.
- **Metódusok:**
 - + void advanceTurn(): Eredeti + notifyObservers.
 - **Algoritmus:**
 1. Eredeti algoritmus (snow, vehicles.move, manageTurn).
 2. notifyObservers(„turnAdvanced”).
 - + void endGame(): Eredeti + notifyObservers.

- **Algoritmus:**

1. Eredeti algoritmus (rangsor + győztes).
2. notifyObservers(„gameEnded”).

30. HomeBase – módosult

- **Felelősség:** A telephelyen történő dokkolás, fejcsere és tankolás műveletek végén notifyObservers fut.
- **Ősosztályok:** Building (változatlan), Observable (új mellék-ős)
- **Interfészek:** Field (változatlan)
- **Attribútumok:**
 - Nincs.
- **Metódusok:**
 - + void acceptSnowPlow(SnowPlow sp): Eredeti + notifyObservers.
 - **Algoritmus:**
 1. Eredeti algoritmus (kapacitás + dockedPlows).
 2. notifyObservers(„plowDocked”).
 - + void swapHead(SnowPlow sp, CleanerHead newHead): Eredeti + notifyObservers.
 - **Algoritmus:**
 1. Eredeti algoritmus (sp.changeHead).
 2. notifyObservers(„headSwapped”).
 - + void refuelSnowPlow(SnowPlow sp, double amount): Eredeti + notifyObservers.
 - **Algoritmus:**
 1. Eredeti algoritmus (activeHead.refillFuel).
 2. notifyObservers(„refuel”).

31. IntegratedMarket – módosult

- **Felelősség:** A buyItem siker / hibás ágán notifyObservers-t hív.
- **Ősosztályok:** Observable (új ős)
- **Interfészek:** Nincs
- **Attribútumok:**
 - Nincs.
- **Metódusok:**
 - + boolean buyItem(Player buyer, IPurchasable item): Eredeti + notifyObservers.
 - **Algoritmus:**
 1. Eredeti algoritmus (egyenleg-ellenőrzés, money csökkentés).
 2. HA siker, AKKOR notifyObservers(„itemBought”).
 3. KÜLÖNBEN notifyObservers(„itemUnaffordable”).

32. Map – módosult

- **Felelősség:** Egyetlen új attribútum (layout) és egy új metódus (loadLayout). A modell-logika változatlan: BFS, snow(), findShortestPath.
- **Ősosztályok:** Nincs (változatlan)
- **Interfészek:** Nincs
- **Attribútumok:**
 - + layout: MapLayout – A pálya képernyő-elrendezése (ÚJ).
- **Metódusok:**
 - + void loadLayout(String filename): Külön fájlból tölti a layoutot.

▪ **Algoritmus:**

1. layout.loadFromFile(filename) hívása.

33. SaveCommand / LoadCommand – módosult

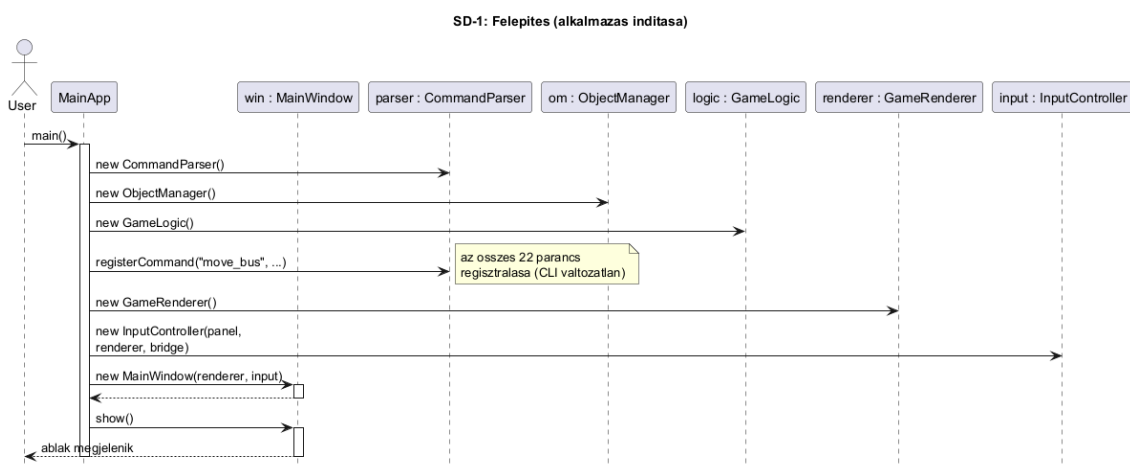
- **Felelősség:** A meglévő save kimenetét kibővítjük: a fájl végén kiírja a set_position parancsokat is, ha a Map.layout-ban van regisztrált pozíció.
 - **Össztályok:** Nincs (változatlan)
 - **Interfészek:** ICommand (változatlan)
 - **Attribútumok:**
 - Nincs.
 - **Metódusok:**
 - + void execute(String[] args): Eredeti + a végén (save) a layout-pozíciók kiírása; az új (load) bemenetkor a SetPositionCommand-on keresztül a layout feltöltése.
- **Algoritmus:**
1. Eredeti algoritmus (save: create / spawn / connect_fields kiírása).
 2. HA Map.layout nem üres, AKKOR minden pozíció set_position parancsként kiírva.
 3. Load oldalon a SetPositionCommand a layout-ot tölti.

11.4 Kapcsolat az alkalmazói rendszerrel

Az alábbi szekvencia-diagramok mutatják, hogyan épül fel és kapcsolódik össze a grafikus rendszer a meglévő rendszerrel, hogyan jutnak el a felhasználó akciói a modellig, és hogyan kerül vissza a kép a képernyőre.

11.4.1 Felépítés (alkalmazás indítása)

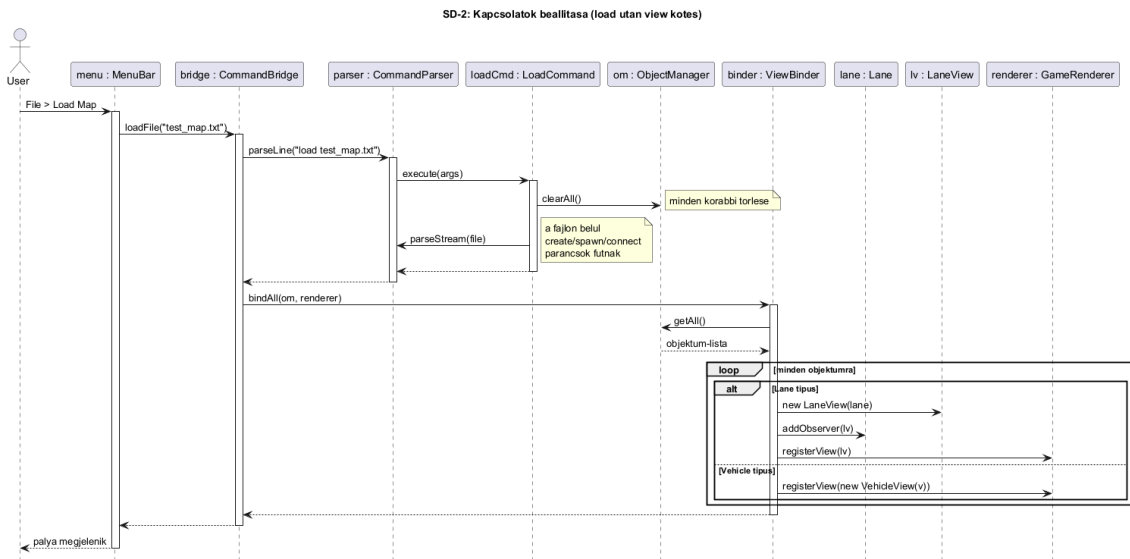
Az alkalmazás indulásakor a MainApp.main() ugyanazt a CLI-rétegbeli objektum-felépítést végzi el, mint a ProtoApp; ezután hozza létre a rendert, az inputot és a főablakot.



11.10. ábra – SD-1: alkalmazás indítása.

11.4.2 Kapcsolatok beállítása (load utáni view-kötés)

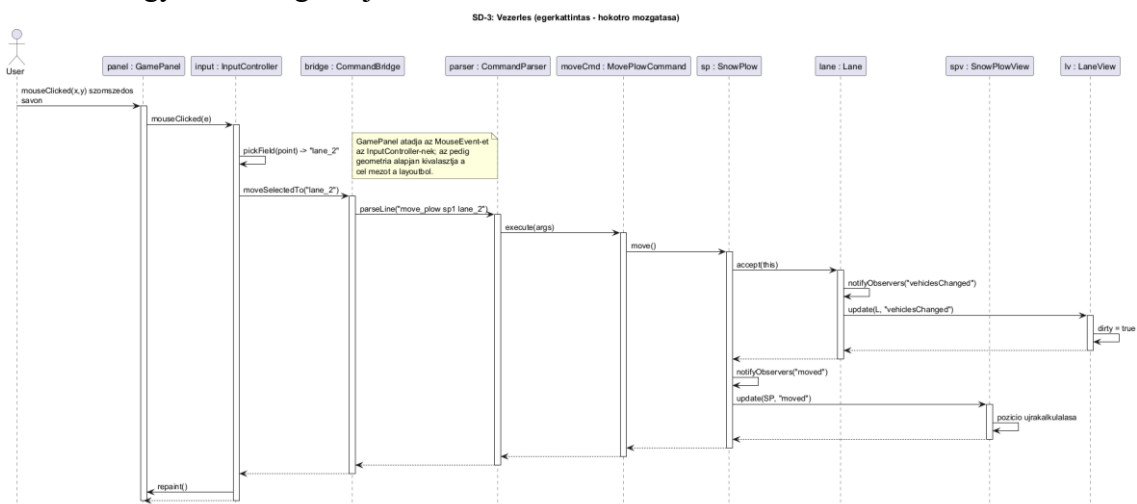
Egy load után a ViewBinder.bindAll iterál az ObjectManager összes elemén; minden modell-elemhez létrehoz egy view-t, felveszi a megfigyelői láncba, és regisztrálja a renderer-be.



11.11. ábra – SD-2: load utáni view-kötés.

11.4.3 Vezérlés (egér-kattintás → modell)

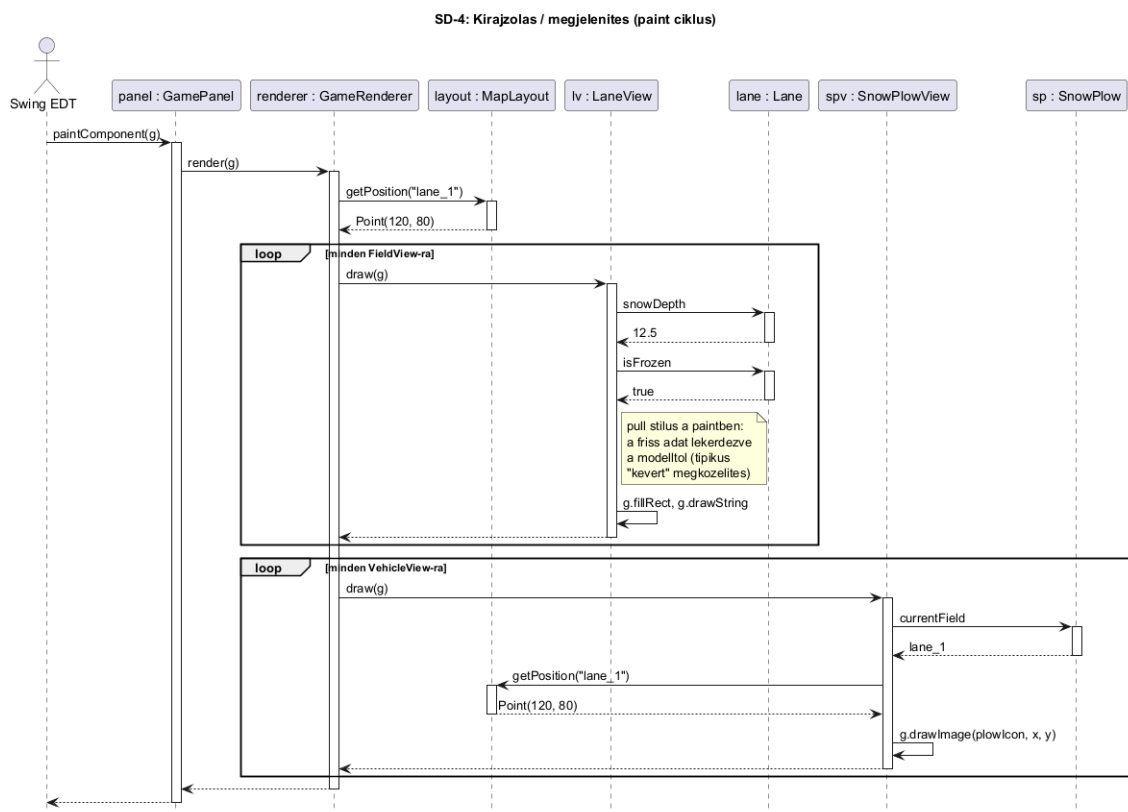
Egy felhasználói kattintás (vagy billentyű) hatására az InputController kiválasztja a célmezőt, a CommandBridge felépíti a megfelelő parancsstringet, majd a parser → ICommand → modell úton ugyanaz a végrehajtás történik, mint a CLI esetén.



11.12. ábra – SD-3: hókotró mozgása egér-kattintással.

11.4.4 Kirajzolás / megjelenítés

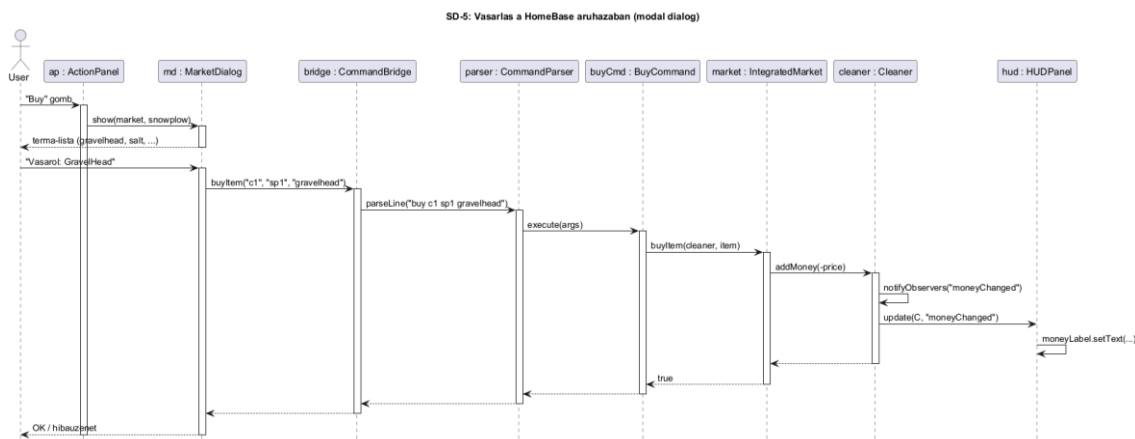
A Swing event dispatch thread-en belül a paintComponent meghívja a renderer.render(g)-t, az pedig egy fix sorrendben (utak, sávok, épületek, járművek) végigmegy a regisztrált nézeteken.



11.13. ábra – SD-4: paint cycle.

11.4.5 Vásárlás a HomeBase áruházában

A MarketDialog modálisan jelenik meg a kijelölt SnowPlow-ra. A felhasználó által választott termékhez a MarketDialog felépíti a buy parancsot, az IntegratedMarket lefolytatja a tranzakciót, és a Cleaner.addMoney(-price) jelzésére a HUDPanel azonnal frissül.



11.14. ábra – SD-5: vásárlás folyamata.

11.5 Napló

Kezdet	Időtartam	Résztevők	Leírás
2026.04.24. 18:00	2 óra	Kálmán, Hája, Papp, Gróf, Dobronyi	Értekezlet. Döntés: a GUI-t Swing-re alapozzuk; az MVC tervezésnél push-alapú Observer mintát választunk, hogy a modellt minimálisan kelljen módosítani. Hája és Kálmán átnézte a

			7-8. heti modellt és összegyűjtötte, mely osztályokon kell notifyObservers-t hozzáadni.
2026.04.25. 19:00	3 óra	Kálmán	Tevékenység: 11.1 fejezet főképernyő-mockupjainak elkészítése (főmenü, játékpálya, áruház, end-game). Az ikon-jelölések rögzítése (Lane / Road / Building / Car / Bus / SnowPlow).
2026.04.26. 14:00	4 óra	Papp	Tevékenység: 11.2.1 (működési elv) és 11.2.2 (osztálystruktúra) szövegezése; az új view csomag (FieldView/VehicleView hierarchia, GameRenderer, MapLayout) terveinek kidolgozása.
2026.04.27. 16:00	3 óra	Hája	Tevékenység: 11.3.1 új osztályok katalogizálása (MainWindow, GamePanel, HUDPanel, ActionPanel, MarketDialog, EndGameDialog), minden metódusra felelősség- és algoritmus-leírás.
2026.04.28. 18:00	3 óra	Gróf	Tevékenység: 11.3.1 controller-osztályok kidolgozása (InputController, CommandBridge, ViewBinder, MainApp), és 11.3.2 modellbeli változások listájának összeállítása (Observable bevezetése, notifyObservers-hívások helye).
2026.04.29. 19:00	3 óra	Dobronyi	Tevékenység: 11.4 szekvencia-diagramok elkészítése (felépítés, load utáni view-kötés, vezérlés, paint cycle, vásárlás) PlantUML-ben.
2026.04.30. 18:00	2 óra	Kálmán, Hája, Papp, Gróf, Dobronyi	Értekezlet. Döntés: az új layout-formátum (set_position) opcionális, így a CLI tesztcsomag (RunAllTests + 20 db _in.txt) változatlan marad. Döntés: a save kimenetét kibővítjük a set_position sorokkal is, ha a Map.layout-ban van adat.
2026.05.02. 17:00	3 óra	Kálmán	Tevékenység: az 5 db szekvencia-diagram és az 5 db osztály-/áttekintő diagram átnézése a katalógussal való konzisztenciára; a feladatban szereplő minden metódus megjelenése valamelyik diagramon.
2026.05.05. 18:00	2 óra	Papp	Tevékenység: a dokumentum összeszerelése a templ_11_0 sablon alapján (11.1, 11.2, 11.3, 11.4, 11.5 fejezetek), formázás.
2026.05.07. 18:00	1 óra	Hája	Tevékenység: 11.1.3 vezérlés-leírás kibővítése a 7. heti use-case lista (1-

			15) lefedettségének ellenőrzésével.
2026.05.08. 19:00	1 óra	Kálmán, Hája, Papp, Gróf, Dobronyi	Értekezlet. Döntés: a dokumentum véglegesítése; minden szakaszt átolvastunk, a katalógus és a szekvenciák konzisztensek. A leadott verzió a beadási mappában a 48_week11.pdf.